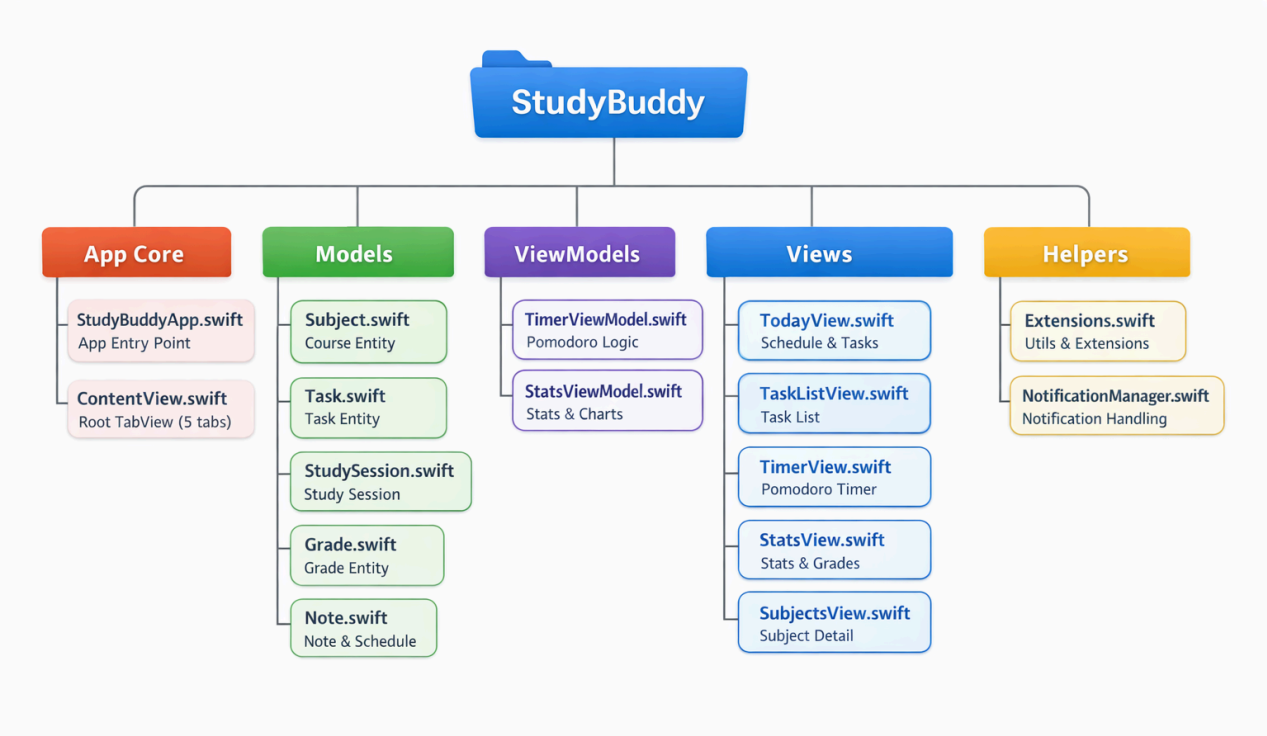
DEVELOPER'S GUIDE (Design Document)

1. Project Overview

StudyBuddy is an iOS study tracking application built with SwiftUI and SwiftData. It helps students manage their timetable, tasks, study sessions, grades, and notes in one place.

Platform: iOS 17+
Language: Swift 5.9
Framework: SwiftUI + SwiftData
Architecture: MVVM (Model-View-ViewModel)
Minimum Xcode: 15.0

2. Project Structure



3. Database Layer — SwiftData

How it works: SwiftData is Apple's modern ORM built on top of Core Data. Each `@Model` class maps directly to a database table. Properties become columns automatically.

Container setup — `StudyBuddyApp.swift`:

```
swift
let container: ModelContainer

init() {
    container = try ModelContainer(for:
        Subject.self, Task.self, StudySession.self,
        Grade.self, Note.self, ScheduleItem.self
    )
}
```

The container is created **once** in `init()` and shared via `.modelContainer(container)`. This is critical — creating it inline with `.modelContainer(for: [...])` can produce separate instances for sheets vs the main window, causing inserts to silently disappear.

The 6 model classes:

Model	Purpose	Key Fields
Subject	A course/subject	name, colorHex, icon, scheduleType scheduleType
Task	Homework, exam, project	title, dueDate, priority, type, isCompleted
Study Session	One timed study block	startTime, endTime, durationSeconds, isPomodoro
Grade	A scored assessment	title, score, maxScore, gradeType
Note	Free-text note per subject	title, content, updatedAt
Schedule Item	Recurring class/block	weekday, startTime, endTime, scheduleWeek

Relationships use `@Relationship(deleteRule: .cascade)` — deleting a Subject also deletes all its tasks, sessions, grades, notes, and schedule items.

Querying uses `@Query` in views:

Swift:

```
@Query(sort: \Subject.name) private var subjects: [Subject]
```

SwiftData observes changes and re-renders the view automatically.

Saving:

Swift

```
modelContext.insert(newObject)

try modelContext.save()
```

4. ViewModel Layer

`TimerViewModel.swift` — an `@Observable` class that owns the Pomodoro state machine:

- `start()` → creates a `StudySession` and inserts it into the context, then fires a `Timer.scheduledTimer` every second
- `tick()` → decrements `secondsRemaining`, increments `secondsElapsed`, updates `currentSession.durationSeconds` live
- `phaseCompleted()` → fires haptics + local notification, saves the session, calls `advancePhase()`
- `advancePhase()` → cycles Focus → Short Break → Long Break → Focus based on `sessionCount`
- `stop()` / `pause()` → stops the timer and saves the partial session if it ran for more than 5 seconds

The ViewModel receives `modelContext` via `setContext(_ context: ModelContext)` called from `TimerView.onAppear`. This avoids the sheet context bug — it uses the view's context directly.

`StatsViewModel.swift` — a pure computation class with no stored state:

- `currentStreak(sessions:)` → walks backwards from today counting consecutive days with at least one session
- `dailyStudyData(sessions:)` → returns 7 `DayStudyData` structs for the bar chart
- `subjectBreakdown(sessions:)` → groups sessions by subject, calculates percentage of weekly total
- `peakHour(sessions:)` → finds the hour of day with the most total study seconds

5. Animation Inventory

Location	Animation	Code
25:00 Timer ring	Smooth countdown	`.animation(.linear(duration: 1), value: vm.progress)`
Task complete	Spring check	`withAnimation { task.isCompleted.toggle() }`
12 Streak 25 Tasks This Week	Slide-in on appear	`.slideIn(delay: 0.05)` — custom ViewModifier Custom ViewModifier
12 Streak 25 Tasks This Week	Counting number	AnimatedNumber view with `.contentTransition(.numericText())`
Math 5	Colour crossfade	`withAnimation(.easeInOut(duration: 0.2))`
Low Med High	Spring highlight	`.animation(.spring(response: 0.3, dampingFraction: 0.7))`
	Scale bounce	`.scaleEffect(selectedIcon == icon ? 1.1 : 1.0) + + .animation(.spring(...))`
List changes	Fade	`.animation(.easeInOut, value: subjects.count)`

6. Notification System — `NotificationManager.swift`

Singleton (`NotificationManager.shared`) wrapping `UNUserNotificationCenter`:

- `requestPermission()` — called on first launch from `ContentView.onAppear`
- `scheduleTaskReminder(for task:)` — creates a calendar trigger for `task.reminderDate`
- `cancelTaskReminder(for task:)` — removes pending notification when task is completed

- `sendPhaseCompleteNotification(phase:)` — fires immediately (0.5s delay) when Pomodoro phase ends
- `scheduleStreakReminder()` — daily 8 PM repeating notification to keep the streak alive

7. Rotation Schedule Logic

`ScheduleType` enum has four cases: `.weekly`, `.aWeek`, `.bWeek`, `.both`.

In `TodayView`, the schedule is filtered like this:

swift

```
let parity = Calendar.current.currentWeekParity  // .aWeek or .bWeek based on weekOfYear % 2
return allScheduleItems.filter { item in
    guard item.weekday == weekday else { return false }
    switch item.scheduleWeek {
    case .weekly, .both: return true
    case .aWeek: return parity == .aWeek
    case .bWeek: return parity == .bWeek
    }
}
```

8. Key Bug Fixed — SwiftData Sheet Context

Problem: `.sheet { }` in SwiftUI can receive a different `ModelContext` from the environment than the parent view uses for its `@Query`. Inserts in the sheet go into a separate context, so the list never updates.

Fix: Create `ModelContainer` once in `App.init()` as a `let` property, then pass it via `.modelContainer(container)`. This ensures every view — including sheets — shares one context instance.

9. How to Run

1. Open `StudyBuddy.xcodeproj` in Xcode 15+
2. Select your team under Signing & Capabilities
3. Choose iPhone 15 simulator (iOS 17+) or a real device
4. Press Run (⌘R)
5. No external dependencies or package manager required.